Report: Route Blocking for Reducing Forest Fire Spread

COL761 Assignment 2, Q2

Parth Singh – 2022ME11438 Anand Sasikumar – 2022CS11087

1 Task 1: Optimization Formulation

We model the route blocking problem as a combinatorial optimization problem over the edge set of a directed probabilistic graph $G = (V, E, p)$.

Decision Variables. For each directed edge $e = (u, v) \in E$, define $x_e \in \{0, 1\}$, where $x_e = 1$ if edge e is blocked and $x_e = 0$ otherwise. The blocked set is $R = \{e \in E : x_e = 1\}$.

Objective Function. Let $G_R = (V, E \setminus R, p)$ be the modified graph. Under the Independent Cascade (IC) model on G_R, let A_∞ be the set of all nodes eventually burned from seed set A_0. The objective is: $\min_x \ \sigma(R) = \mathbb{E}[|A_\infty|]$, where the expectation is over the stochastic fire spread (each edge (u, v) independently transmits fire with probability p_{uv}).

Constraints. (1) Budget: $\sum_{e \in E} x_e = k$. (2) Binary: $x_e \in \{0, 1\}$ for all $e \in E$. For the h-hop variant, the cascade is restricted so that no node v with $d(A_0, v) > h$ can burn.

2 Task 2: NP-Hardness

Theorem 1. *The route blocking problem is NP-hard.*

Proof. We reduce from the NP-hard Set Cover problem. Given universe $U = \{u_1, \ldots, u_n\}$ and collection $\mathcal{S} = \{S_1, \ldots, S_m\}$, the task is to find at most k sets covering U.

Construction. Build directed graph G: (i) Create source s (sole seed, $A_0 = \{s\}$). (ii) For each S_j, create node v_j and edge (s, v_j) with $p = 1$. (iii) For each $u_i \in U$, create node w_i. (iv) For each $u_i \in S_j$, add edge (v_j, w_i) with $p = 1$.

Analysis. Since all $p = 1$, the cascade is deterministic. With no blocking, $\sigma(\emptyset) = 1 + m + n$. Each blocked edge (s, v_j) prevents v_j and its exclusively-covered elements from burning. Blocking k edges from $\{(s, v_j)\}$ is equivalent to choosing k sets; $\sigma(R)$ is minimised when the chosen sets cover the most elements. If a set cover of size k exists, then $\sigma(R) = 1 + (m - k)$. Deciding whether $\sigma(R) \leq 1 + (m - k)$ is equivalent to Set Cover. The reduction is polynomial. □ □

3 Task 3: Structural Properties

Define the reduction function $f(R) = \sigma(\emptyset) - \sigma(R)$.

3.1 Monotonicity

Claim 1. *$f(R)$ is monotone non-decreasing: for $R \subseteq R'$, $f(R) \leq f(R')$.*

Proof. We show $\sigma(R) \geq \sigma(R')$ whenever $R \subseteq R'$. Fix any realisation ω of the stochastic process. In $G_{R'}$, strictly fewer edges are present than in G_R (since $R \subseteq R'$), so the set of nodes reachable from A_0 in $G_{R'}$ is a subset of those reachable in G_R. This holds for every ω, so $\mathbb{E}[|A_\infty(R')|] \leq \mathbb{E}[|A_\infty(R)|]$, giving $f(R') \geq f(R)$. □

3.2 Submodularity

Claim 2. *$f(R)$ is submodular: for $R \subseteq S$ and $e \notin S$, $f(R \cup \{e\}) - f(R) \geq f(S \cup \{e\}) - f(S)$.*

Proof. Fix realisation ω; let $\text{Reach}(R, \omega)$ be nodes reachable from A_0 via live edges not in R. For $R \subseteq S$: $\text{Reach}(S, \omega) \subseteq \text{Reach}(R, \omega)$.

Adding $e = (u, v)$ to the blocked set reduces the reachable set only if: (a) $u \in \text{Reach}$, (b) e is live, and (c) some node reachable through v has no alternative path. Since $\text{Reach}(R, \omega) \supseteq \text{Reach}(S, \omega)$, condition (a) is at least as likely under R, and the larger reachable set means condition (c) also holds at least as often (fewer alternative paths have been cut). The set of nodes "saved" by blocking e under R is thus a superset of those saved under S, for every ω. Taking expectations yields the submodularity inequality. □

Remark. Both properties hold identically in the h-hop variant, since hop restriction only further constrains reachability.

4 Task 4: Algorithm Design

4.1 Algorithm Overview

We propose a three-phase algorithm: seed-edge prioritisation, CELF greedy, and local search.

1. **Phase 0 – Seed-Edge Prioritisation.** Rank all outgoing edges from seeds A_0 by probability p descending. Block top-k as initial solution. This exploits the bottleneck: cutting edges at the source prevents all downstream spread.
2. **Phase 1 – CELF Greedy.** Score candidates by $\text{score}(e) = P(\text{fire reaches } u) \cdot p_{uv} \cdot (1 + \text{out-deg}(v)) \cdot$ seed-bonus. Evaluate marginal gains via Monte Carlo, then run CELF [**?**]: iteratively select the edge with highest marginal gain using lazy evaluation.
3. **Phase 2 – Local Search.** For each blocked edge, try swapping with the best unblocked candidate. Accept if spread decreases. Repeat until convergence or time limit.

The better of Phase 0 and Phase 1 is kept, then refined by Phase 2.

4.2 Pseudocode

Algorithm 1 Seed-Edge CELF with Local Search

Require: $G = (V, E, p)$, seeds A_0, budget k, sims N, hops h
Ensure: Blocked set R, $|R| = k$

Sort edges from A_0 by p desc; $R_0 \leftarrow$ top-k; write to output
Score all edges; select top-C candidates
for each candidate e_i **do** $g_i \leftarrow \sigma(\emptyset) - \sigma(\{e_i\})$ via N simulations
end for
Init max-heap H with $(g_i, i, 0)$
for $t = 1$ to k **do**
    repeat pop (g, i, τ) from H
        if $\tau = t$ **then** add e_i to R_1; update baseline; **break**
        else re-evaluate; push (g_i', i, t)
        end if
    until accepted or heap empty
end for
$R \leftarrow \arg\min\big(\sigma(R_0), \sigma(R_1)\big)$
for each $e \in R$ **do** ▷ Local search
    Find e' s.t. $\sigma((R \setminus \{e\}) \cup \{e'\}) < \sigma(R)$; if found, swap
end for
return R

4.3 Computational Complexity

Let $m = |E|$, $C =$ candidates, $N =$ simulations, $P =$ replacement pool size. Phase 0 is $O(m \log m)$. Phase 1 initialisation: $O(C \cdot N \cdot m)$; CELF iterations: $O(k \cdot N \cdot m)$ amortised. Phase 2: $O(k \cdot P \cdot N \cdot m)$ per pass. Overall: $O((C + kP) \cdot N \cdot m)$.

4.4 Approximation Guarantee

Since $f(R)$ is monotone submodular (Task 3), the greedy algorithm achieves $(1 - 1/e) \approx 0.632$ approximation [**?**]: $f(R_{\text{greedy}}) \geq (1 - 1/e) \cdot f(R_{\text{OPT}})$. CELF preserves this guarantee via lazy evaluation [**?**]. Candidate pre-filtering is a heuristic that may forfeit the formal guarantee if the optimal edge is excluded, but in practice the scoring reliably identifies all high-impact edges.

5 Task 5: Competitive Evaluation Results

We evaluate using the official `evaluate.py` with `num_sim` $= 50$.

Dataset	k	hops	$\sigma(\emptyset)$	$\sigma(R)$	$\rho(R)$	Time
1 (1 seed)	50	-1 (unlimited)	36.28	9.12	**0.7486**	< 1s
2 (10 seeds)	20	3	124.42	14.84	**0.8807**	< 1s

Table 1: Evaluation results on public datasets.

Dataset 1. Seed node 101 has 64 outgoing edges. With $k = 50$, blocking the 50 highest-probability edges severs most propagation paths ($\rho \approx 75\%$). The seed-edge strategy dominates because the single seed is the sole bottleneck.

Dataset 2. With 10 seeds and $h = 3$, the reachable set is $\sim$383 nodes. Key bottlenecks are high-degree seeds (node 314: 5 out-edges reaching 200 nodes; node 13925: 107 reachable nodes). CELF greedy accounts for diminishing returns across overlapping neighbourhoods, achieving $\rho \approx 88\%$.

Key insight. A naive heuristic based on edge probability and local out-degree yielded $\rho = 0.117$ on Dataset 1, wasting budget on leaf-node edges. The seed-focused strategy improves ρ by 6.4$\times$.